

Instrukcja obsługi projektu

Ten projekt to wizualizacja 3D linii produkcyjnej paczkomatu. Działa w przeglądarce. Poniżej znajdziesz wszystko, co trzeba wiedzieć, żeby go uruchomić i rozwijać.

Jak uruchomić projekt na swoim komputerze

1. Zainstaluj Node.js, jeśli jeszcze go nie masz. Pobierz go ze strony <https://nodejs.org> (wybierz wersję LTS) i zainstaluj jak zwykły program.
2. Otwórz terminal w folderze projektu. Na Windows: otwórz folder projektu w Eksploratorze, kliknij prawym przyciskiem w pustym miejscu i wybierz "Otwórz w terminalu".
3. Wpisz po kolei te dwa polecenia (każde zatwierdź Enterem):

```
npm install  
npm run dev
```

Pierwsze pobiera potrzebne biblioteki (robi się to tylko raz). Drugie uruchamia aplikację.

4. Otwórz w przeglądarce adres, który pojawi się w terminalu, zazwyczaj: `http://127.0.0.1:5173`

Żeby zatrzymać aplikację, wróć do terminala i naciśnij Ctrl+C.

Gdzie zmieniać kod

Najważniejsze pliki:

- `src/main.jsx` - interfejs aplikacji i scena 3D. Tu jest większość kodu.
- `src/simulation.js` - obliczenia harmonogramu linii (czasy, takty, kolejność).
- `src/styles.css` - wygląd (kolory, czcionki, rozmieszczenie paneli).
- `public/config/strefy.json` - układ stref hali. Nie trzeba go edytować ręcznie, służy do tego edytor pod adresem `/edytor_stref.html` (przycisk z ołówkiem w aplikacji).
- `public/dokumentacja.html` - dokumentacja dla inżynierów, dostępna w aplikacji pod przyciskiem "?".

Gdy aplikacja działa przez `npm run dev`, każda zapisana zmiana w kodzie jest od razu widoczna w przeglądarce. Nie trzeba nic restartować.

Po większych zmianach warto uruchomić testy poleceniem `npm test`. Jeśli na końcu zobaczysz "0 bledow", wszystko jest w porządku.

Gdzie aplikacja jest hostowana

Działająca wersja jest obecnie hostowana na moim prywatnym koncie Vercel:

<https://linia-produkcyjna-paczkomat.vercel.app>

Vercel sam buduje i publikuje aplikację. Jeśli ta wersja Wam wystarcza, nie musicie nic robić.

Jak postawić własny hosting

Jeśli chcecie mieć aplikację na własnym koncie, najprościej zrobić to tak:

1. Załóżcie darmowe konto na <https://vercel.com> (można zalogować się kontem GitHub).
2. Wrzućcie folder projektu jako repozytorium na <https://github.com> (bez folderu `node_modules`, jest niepotrzebny i bardzo duży).
3. W panelu Vercel kliknijcie "Add New" i "Project", potem wybierzcie swoje repozytorium z listy i kliknijcie "Deploy".
4. Vercel sam wykryje ustawienia (są w pliku `vercel.json`) i po chwili poda adres, pod którym działa aplikacja.

Od tej pory każda zmiana wgrana do repozytorium na GitHub będzie automatycznie publikowana pod tym adresem.

Aplikacja nie ma bazy danych ani własnego serwera, to zwykła strona statyczna. Zadziała więc też na każdym innym hostingu stron. Wystarczy zbudować ją poleceniem `npm run build` i wgrać na serwer zawartość folderu `dist`, który wtedy powstanie.

Jak powstawał projekt

Do programowania używałem modeli językowych: Claude Fable 5, Claude Sonnet 5, Claude Opus 4.8 oraz ChatGPT 5.5.

Na obecną chwilę nawet flagowe LLM-y nie radzą sobie dobrze z animowaniem i ustawianiem obiektów w scenie 3D. Dlatego całe pozycjonowanie części, dopasowanie animacji montażu i strojenie sceny robiłem ręcznie, metodą prób i poprawek.

Modele 3D realnego paczkomatu, które widać w aplikacji, to pliki GLB. Powstały z plików CAD-owskich, które konwertowałem w Blenderze z pomocą rozszerzenia potrafiącego odczytywać takie pliki. Z całego modelu ręcznie wyodrębniałem osobne moduły (koryta, drzwi, ściany, dach itd.) i każdy eksportowałem jako oddzielny plik. Dzięki temu aplikacja może pokazywać montaż części po części.

Surowe pliki z CAD-a były za ciężkie do strony internetowej, więc przy eksporcie stosowałem kilka metod zmniejszania (downscalingu), między innymi:

- redukcję liczby wielokątów siatki (w Blenderze modyfikator Decimate), bo modele CAD mają o wiele więcej detali, niż potrzeba na ekranie,
- usuwanie geometrii niewidocznej z zewnątrz, np. śrub, gwintów i wewnętrznych elementów konstrukcyjnych,
- scalanie zdublowanych wierzchołków (Merge by Distance), które zostają po konwersji z CAD-a,
- kompresję Draco przy eksporcie do GLB (dlatego w projekcie jest folder `public/draco` z dekoderm, który rozpakowuje modele w przeglądarce),
- rezygnację z tekstur na rzecz prostych kolorów materiałów, bo tekstury ważą najwięcej, a przy tej skali i tak ich nie widać.